Studio Nx

Backend(FastAPI/Python) 42 — POST · DELETE /api/v1/users/me/profile-image

2026-10-12

Backend(FastAPI/Python) 42 — `POST · DELETE /api/v1/users/me/profile-image`

항목	내용
파일	`studionx/backend/app/api/v1/users.py`의 `upload_profile_image()`, `delete_profile_image()`
작업	P1-08 회원 정보: 프로필 사진 올리기와 지우기 (PRD 01 FR-106)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	41 GET · PATCH /users/me · 43 GET /profile-images/{name}

1. 하는 일

요청	응답
`POST` multipart, `file` 칸에 JPG(1.2MB)	200과 내 정보입니다. `"profile_image_url": "/api/v1/profile-images/3f9c….jpg"`
`POST` GIF	415 `UNSUPPORTED_FILE_TYPE`
`POST` 2MB 초과	413 `FILE_TOO_LARGE`
`POST` 칸 이름이 `file`이 아님 (`photo`)	422 `VALIDATION_ERROR`
`DELETE`	200과 내 정보입니다. `"profile_image_url": null`이고, 파일이 지워집니다
`DELETE` (사진 없음, 두 번 누름)	200
토큰 없음	401이고, 파일을 저장하지 않습니다

2. 언제 쓰나

화면 (userInfo.html, P1-10)	부르는 것
Upload photo → 파일 고르기	`FormData`에 `file`을 넣어 `api.post('/users/me/profile-image', formData)`로 보내고, 응답의 `profile_image_url`로 `<img>`를 바꿉니다
Remove	`api.delete('/users/me/profile-image')`로 보내고, 첫 글자 동그라미로 돌아갑니다

함수들이 이어지는 순서 (`POST`)

순서	처리하는 곳	하는 일
①	06 `CurrentUser`	토큰으로 활동 회원을 찾습니다 (아니면 401이고, 파일은 읽지도 않습니다)
②	FastAPI `UploadFile`	multipart의 `file` 칸을 받습니다 (없으면 422)
③	38 `save_profile_image`	종류, 크기, 내용을 검사하고 저장한 뒤 예전 파일을 지웁니다
④	36 `to_me_response`	새 사진 주소(35)가 담긴 응답을 만듭니다

3. 코드

import (파일 맨 위: 41번과 같고, 새로 추가한 것은 없습니다)

함수 (경로와 응답 설정은 템플릿 그대로입니다)

```
@router.post(
    "/me/profile-image",
    response_model=MeResponse,
    responses={413: {"model": ErrorResponse}, 415: {"model": ErrorResponse}},
)
async def upload_profile_image(file: UploadFile, user: CurrentUser, db: Db) -> MeResponse:
    user = await user_service.save_profile_image(db, user, file)
    return user_service.to_me_response(user)


@router.delete("/me/profile-image", response_model=MeResponse)
def delete_profile_image(user: CurrentUser, db: Db) -> MeResponse:
    return user_service.to_me_response(user_service.delete_profile_image(db, user))
```

문법 풀이

식	뜻
`file: UploadFile`	multipart 본문의 `file` 칸을 파일로 받습니다. 큰 파일은 디스크의 임시 파일에 두어서 메모리를 아낍니다. `python-multipart` 패키지가 필요합니다 (requirements에 있습니다)
`async def`, `await`	38번이 `await file.read(…)`를 쓰기 때문에 라우터도 비동기로 만듭니다
`user = await …`	매개변수 `user`에 저장 결과(같은 회원, 새 경로)를 다시 넣습니다

4. 흐름

POST: 토큰을 확인하고, `file` 칸을 받은 뒤, 38번으로 검사하고 저장한 다음 36번으로 응답합니다.

DELETE: 토큰을 확인하고, 39번으로 지운 뒤 36번으로 응답합니다.

5. 왜 이렇게

선택	이유
라우터는 연결만 함	검사, 저장 순서, 실패 처리는 38번과 39번에서 이미 시험을 마쳤습니다
응답은 내 정보 전체	화면이 새 사진 주소를 바로 받아서 `<img src>`를 바꿀 수 있습니다 (다시 `GET`을 보낼 필요가 없습니다)
사진이 없어도 `DELETE`는 200	39번과 같은 이유입니다. 두 번 눌러도 사용자에게 오류를 보여 주지 않습니다
토큰 검사를 파일보다 먼저 함	로그인하지 않은 사람이 파일을 올려도 저장되지 않습니다 (시험으로 확인했습니다: `profiles` 폴더가 생기지도 않습니다)

알아 둘 점 (비동기 함수 안의 DB 작업): `upload_profile_image`는 `async` 함수인데, 38번 안의 DB 저장(`commit`)은 동기 방식입니다. 그래서 저장하는 동안 서버의 다른 요청이 아주 잠깐 기다립니다. 사진 저장은 한 번에 수 밀리초 정도라서 Phase 1에서는 문제가 없습니다. 사용자가 많아지면 파일만 먼저 읽고, DB 작업은 동기 함수로 나누면 됩니다.

6. 시험 (`tests/test_user_routes.py`에 7개를 추가해서 모두 18개, 실제 앱과 MySQL, 임시 폴더로 실행)

시험	확인한 것
JPG 올리기	200이고, 주소 모양이 `/api/v1/profile-images/{32자}.jpg`이며, 그 파일의 내용이 올린 것과 같습니다
GIF, 2MB 초과	415 `UNSUPPORTED_FILE_TYPE`, 413 `FILE_TOO_LARGE`
칸 이름 `photo`	422 `VALIDATION_ERROR`
토큰 없이 올리기	401이고, 저장 폴더가 생기지도 않습니다
지우기	200이고, 주소는 `null`이며 파일이 지워집니다. 한 번 더 지워도 200입니다
토큰 없이 지우기	401

전체 시험 306개가 통과했고, `ruff` 검사도 통과했습니다. 실제 `storage/` 폴더는 비어 있습니다 (`.gitkeep`만 있습니다).